Guía práctica de Git y GitHub

Manual reorganizado: desde la primera configuración hasta ramas, Lazygit y recuperación de versiones.

Edición: octubre de 2026 · Windows / Git Bash · Ejemplos de tesis, PLANIALTIS y proyectos LaTeX.

Cómo usarla: empezá por el caso de uso de la sección 2. Los capítulos siguientes avanzan desde las tareas básicas hasta las operaciones de riesgo.

Conservación del contenido: al final se adjunta íntegra la guía anterior de 48 páginas como archivo de referencia, para no perder ninguno de sus casos, ejemplos y anexos.

Índice de lectura

• 1. Conceptos básicos y comprobaciones

• 2. Primer uso: tres casos distintos

• 3. Conectar una carpeta local con GitHub

• 4. Uso diario: guardar, subir y bajar

• 5. Configuración, identidad y autenticación

• 6. Comprobar qué repositorio y rama estás usando

• 7. Archivos, .gitignore y LaTeX

• 8. Historial, diferencias y etiquetas

• 9. Ramas, merge y seguimiento

• 10. Lazygit: lectura de la interfaz y flujo

• 11. Recuperación segura, stash y worktrees

• 12. GitHub CLI y eliminación de repositorios

• 13. Trabajo entre computadoras y problemas frecuentes

• 14. Tabla de comandos y opciones

• Apéndice: guía anterior completa (48 páginas)

ATENCIÓN: En todos los ejemplos, reemplazá USUARIO, PROYECTO y las rutas por tus valores. No ejecutes ejemplos de borrado, reset o restauración sin comprobar el repositorio y su estado.

1. Conceptos básicos y comprobaciones

Git guarda versiones (commits) en una carpeta local. GitHub aloja un repositorio remoto. Una rama (main o master) es un nombre que avanza con los commits; origin es el alias habitual del remoto; HEAD señala la posición actual. Un tag marca una versión concreta.

```
git --version
pwd
ls -la
git status
git rev-parse --show-toplevel
```

Los comandos de Git funcionan en Git Bash y CMD. Las rutas cambian: Git Bash usa /c/GITHUB/PROYECTO; CMD usa C:\GITHUB\PROYECTO.

2. Primer uso: tres casos distintos

Caso A — La carpeta ya tiene archivos, pero todavía no tiene Git

Abrí Git Bash y entrá en la carpeta. Verificá primero que no esté dentro de otro repositorio. git init -b main crea el repositorio local y elige la rama principal desde el comienzo.

```
cd /c/GITHUB/PROYECTO
git rev-parse --show-toplevel   # Si no hay Git, mostrará error
git init -b main
git status
git add .
```

```
git commit -m "Primera version"
```

Ahora tenés historial local, aunque todavía no existe ningún vínculo con GitHub. Si preferís master, usá git init -b master. Son nombres equivalentes funcionalmente.

Caso B — El repositorio ya existe en GitHub

No ejecutes git init en una carpeta nueva para este caso: git clone descarga el proyecto, configura origin y normalmente deja configurado el seguimiento de la rama principal.

```
mkdir -p /c/GITHUB
cd /c/GITHUB
git clone https://github.com/USUARIO/PROYECTO.git
cd PROYECTO
git status
git remote -v
git branch -vv
```

Para repositorios privados, la cuenta autenticada debe tener acceso; si pertenecen a otra persona, aceptá antes la invitación de colaboración.

Caso C — Git local, sin GitHub

```
cd /c/GITHUB/PROYECTO
git init -b main
git add .
git commit -m "Primera version"
git log --oneline
```

Podés seguir creando commits sin conexión ni remoto. Los commits viven en .git; mantené además una copia de seguridad independiente.

3. Conectar una carpeta local con GitHub

En GitHub, creá un repositorio vacío en github.com/new, elegí nombre y visibilidad. Si ya tenés commits locales, para este procedimiento no marques README, licencia ni .gitignore en la web.

```
cd /c/GITHUB/PROYECTO
git status
git branch --show-current
git remote -v
git remote add origin https://github.com/USUARIO/PROYECTO.git
git push -u origin main
```

remote add origin guarda la URL de destino; push envía commits; -u (set-upstream) asocia la rama local con origin/main para futuros git push y git pull. Si tu rama se llama master, reemplazá main por master.

ATENCIÓN: Si origin ya existe, no repitas remote add. Consultá git remote -v y, solo si la URL es incorrecta, usá git remote set-url origin URL. Si GitHub ya tiene commits, no fuerces el push: primero analizá cómo integrar ambos historiales.

Alternativa con GitHub CLI (gh)

```
gh auth login
gh auth status
gh repo create PROYECTO --private --source=. --remote=origin --push
```

Este procedimiento sustituye la creación manual del repositorio y el remote add: no hagas ambos a ciegas.

4. Uso diario: guardar, subir y bajar

Al empezar a trabajar

```
git status -sb
git pull
```

Guardá tus archivos y revisá si hay cambios locales antes de incorporar cambios remotos. Si no configuraste seguimiento, git pull puede necesitar remoto y rama explícitos.

Al terminar una etapa

```
git status
git diff
git add -A
git diff --staged --stat
git commit -m "Describir el cambio"
git push
```

add prepara; commit registra localmente; push publica. Podés hacer varios commits y luego un solo push. git add -A prepara cambios de todo el repositorio; usalo solo después de revisar qué archivos incluye.

Cómo saber si falta subir algo

```
git fetch origin
git status -sb
git log origin/main..main --oneline
git log main..origin/main --oneline
```

El primer log muestra commits que tenés localmente y no están en origin/main; el segundo, commits de origin/main que aún no integraste. ahead 3 significa tres commits locales pendientes. fetch actualiza referencias sin modificar los archivos de trabajo.

5. Configuración, identidad y autenticación

Una vez por computadora, configurá el nombre y correo que se registrarán en los commits. No son las credenciales de GitHub.

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
git config --global --list
git config --show-origin --list
```

Para sobrescribir la identidad en un solo repositorio:

```
git config --local user.name "Otro Nombre"
git config --local user.email "otro@ejemplo.com"
```

La cuenta autenticada que tiene permiso de push puede ser distinta del autor del commit. Con HTTPS, Git Credential Manager puede gestionar el acceso.

```
git credential-manager github list
git credential-manager github login --username USUARIO
gh auth status
```

El último comando requiere tener GitHub CLI instalado. No compartas contraseñas ni tokens.

6. Comprobar qué repositorio y rama estás usando

```
git rev-parse --show-toplevel
git remote -v
git config --get remote.origin.url
git branch --show-current
git branch -vv
git status -sb
```

rev-parse identifica la raíz local; remote -v muestra URL de fetch y push; branch -vv muestra seguimiento de ramas. origin/main es una referencia local de la última información descargada, no una consulta en vivo a GitHub.

Si querés consultar el destino de push configurado, revisá también git remote get-url --push origin. El destino efectivo de git push puede depender de otras configuraciones, como remote.pushDefault y push.default.

7. Archivos, .gitignore y LaTeX

La carpeta .git guarda el historial. El archivo .gitignore contiene patrones para no incluir auxiliares, copias y temporales. No borres .git.

```
touch .gitignore
notepad .gitignore
ls -a
git check-ignore -v general.aux
```

Ejemplo de contenido de .gitignore para XeLaTeX/latexmk (cada patrón en una línea):

```
*.aux
*.log
*.out
*.toc
*.synctex.gz
*.fls
*.fdb_latexmk
*.xdv
*.bcf
*.run.xml
*.blg
```

Decidí expresamente si querés guardar el PDF final, archivos .bbl y otros resultados de compilación. .gitignore no deja de rastrear archivos ya incorporados; para ello puede utilizarse git rm --cached RUTA, revisando previamente el efecto.

8. Historial, diferencias y etiquetas

```
git log --oneline --graph --all --decorate
git show HASH --stat
git diff
git diff --staged
git diff HASH1 HASH2
git tag
git tag -a V1.00 -m "Version validada"
git push origin V1.00
```

Un commit tiene identificador (hash) y mensaje; una etiqueta marca un commit sin crear otro. Un git push ordinario no garantiza publicar todos los tags. Para archivos binarios como PDF y DWG, Git conserva versiones, pero no ofrece diferencias textuales útiles.

9. Ramas, merge y seguimiento

```
git branch -a
git switch -c pruebas
# Editar, agregar y confirmar cambios
git add -A
git commit -m "Ensayo"
git switch main
git merge pruebas
git branch -d pruebas
```

La rama principal puede llamarse main o master. Una rama nueva apunta inicialmente al commit donde fue creada; los commits posteriores pertenecen a la historia de esa rama, aunque pueden compartir ancestros con otras ramas. El asterisco de git branch indica la rama activa.

Para publicar una rama nueva:

```
git push -u origin pruebas
```

Si dos ramas avanzaron, merge puede generar un commit de fusión o solicitar resolver conflictos. No elimines una rama con -D sin comprobar qué commits exclusivos contiene.

10. Lazygit: lectura de la interfaz y flujo

Lazygit es una interfaz de terminal para Git, distinta de lazy.nvim (administrador de plugins de Neovim). Se inicia desde el repositorio:

```
cd /c/GITHUB/PROYECTO
lazygit
```

En los paneles podés inspeccionar archivos, commits, ramas y remotos. Una flecha hacia arriba junto a una rama indica commits locales por delante de su seguimiento remoto conocido; un número como 3 indica cuántos. Un asterisco suele identificar la rama actual según el panel. Los colores dependen del tema y del estado: verificá la descripción antes de interpretarlos.

Lazygit puede hacer fetch periódico: actualiza información del remoto, pero no integra por sí mismo esos commits en tus archivos. Esc suele cerrar una vista o devolver el foco al panel anterior. Antes de hacer push, revisá la rama activa y el remoto.

11. Recuperación segura, stash y worktrees

Inspeccionar antes de cambiar algo

```
git status -sb
git log --oneline -10
git branch -vv
git worktree list
```

Deshacer un cambio no preparado / retirar del staging

```
git restore --staged archivo.tex
git restore archivo.tex
```

ATENCIÓN: El primer comando conserva las ediciones en disco. El segundo puede descartar ediciones no confirmadas: usalo solo si querés perderlas.

Guardar cambios temporalmente

```
git stash push -u -m "trabajo temporal"
git stash list
git stash pop
```

Abrir una versión histórica sin tocar la actual

```
git worktree add --detach ../PROYECTO_ANTERIOR HASH
git worktree list
# Al terminar, cerrar archivos y ejecutar:
git worktree remove ../PROYECTO_ANTERIOR
```

Deshacer un commit ya publicado sin reescribir historia

```
git revert HASH
git push
```

revert crea un commit inverso; no elimina el historial. Para volver a todo el contenido de una versión antigua, existe git restore --source=HASH --staged --worktree :/ seguido de un commit, pero sobrescribe archivos registrados: respaldá y revisá primero.

ATENCIÓN: git reset --hard mueve la rama y puede destruir cambios locales; git push --force reescribe historia remota. No son soluciones de rutina.

12. GitHub CLI y eliminación de repositorios

GitHub CLI (gh) permite administrar GitHub desde Git Bash. Instalalo y autenticá tu cuenta antes de usarlo.

```
gh auth login
gh auth status
gh repo list
gh repo view
```

Para eliminar un repositorio remoto de GitHub:

```
gh repo delete USUARIO/PROYECTO
```

Desde un directorio Git con remoto de GitHub, gh repo delete puede deducir el repositorio; es más seguro indicar siempre USUARIO/PROYECTO. --yes omite la confirmación interactiva y aumenta el riesgo de borrar el repositorio equivocado.

```
gh repo delete USUARIO/PROYECTO --yes
gh auth refresh -s delete_repo
```

El segundo comando sirve si la autorización de gh necesita el permiso delete_repo. Eliminar un repositorio en GitHub no borra automáticamente tu carpeta local; eliminar la carpeta local tampoco elimina el repositorio remoto. Antes de borrar, verificá gh repo view y git remote -v.

13. Trabajo entre computadoras y problemas frecuentes

En la computadora A: commit y push. En la B: clone la primera vez; después pull antes de editar. Evitá sincronizar una carpeta .git activa entre equipos mediante OneDrive: GitHub debe ser el canal de intercambio de commits.

Mensajes típicos

fatal: not a git repository: estás fuera de un repositorio; usá pwd y git rev-parse --show-toplevel.

remote origin already exists: el remoto ya está configurado; inspeccioná git remote -v.

Repository not found: revisá URL, autenticación y permisos, especialmente en repositorios privados.

rejected / non-fast-forward: el remoto tiene commits no incorporados; hacé fetch, compará y sincronizá antes de publicar.

nothing to commit: no hay cambios preparados nuevos; no es un error.

ahead 3: hay tres commits locales pendientes de publicar, respecto del remoto conocido.

14. Tabla de comandos y opciones

Comando	Uso
git init -b main	Crear Git local y elegir rama
git clone URL	Descargar repositorio y configurar remoto
git remote -v	Ver destinos fetch/push
git branch -vv	Ramas y upstream
git add -A	Preparar cambios de todo el repo
git commit -m "..."	Crear commit local
git push -u origin main	Primer push y seguimiento
git fetch	Actualizar referencias remotas
git pull	Descargar e integrar cambios
git log --oneline	Historial abreviado
git tag	Etiquetas
git worktree list	Copias de trabajo adicionales
git status -sb	Estado corto y rama

Opciones frecuentes: -m = mensaje en commit, pero renombrar en branch; -u = upstream en push; -v = verbose en remote; -vv = más detalle en branch; -c = crear rama en switch; -d = borrar rama fusionada; -D = forzar borrado; --staged = índice; --worktree = archivos físicos; --oneline = resumen de commits. Las opciones dependen del subcomando.

Apéndice documental — edición anterior íntegra

Las siguientes 48 páginas reproducen la guía aportada por el usuario, en su orden original, para conservar íntegramente los ejemplos, los anexos, los casos particulares y las advertencias. El manual reorganizado anterior es la ruta de lectura recomendada.

Algunas rutas, versiones y nombres de repositorio que aparecen en el apéndice son históricos y deben comprobarse antes de ejecutar comandos.

Guía práctica de Git y GitHub

Configuración inicial, creación de repositorios, clonación y sincronización entre computadoras · Con ejemplos para proyectos LaTeX en TeXstudio

1. Instalar Git y configurar la identidad

Instalá Git for Windows desde git-scm.com/downloads/win y abrí Git Bash. Estos comandos se ejecutan una vez por computadora:

```
git config --global user.name "Martin Moran"
git config --global user.email "moran.martin.andres@gmail.com"
git config --global --list
```

El nombre y el correo identifican al autor de cada commit; no son las credenciales para iniciar sesión en GitHub. Para subir a un repositorio privado, la cuenta autenticada debe tener permiso de escritura.

2. Caso A: el repositorio ya existe en GitHub

ANTES DE CLONAR: comprobar el acceso al repositorio privado.

Si es de otra persona, el propietario debe invitarte en GitHub: Settings > Collaborators > Add people. Aceptá la invitación con tu propia cuenta.
Si es tuyo, no necesitás invitación. Si es público, podés clonarlo sin invitación.

Después, para descargarlo por primera vez (ejemplo: Microbiologia):

```
cd /c/BLAS
git clone https://github.com/tinkatul/Microbiologia.git
cd Microbiologia
git status
```

Más detalles sobre invitaciones y permisos: anexo, págs. 14 y 15.

3. Caso B: crear un repositorio propio desde una carpeta existente

Paso 1. Crear un repositorio vacío en GitHub

Entrá en github.com/new, indicá el nombre (por ejemplo, Tesis-Tensegridad) y elegí Private si no querés que sea público. Para esta secuencia, dejá desmarcadas las opciones de crear README, .gitignore y licencia.

Paso 2. Inicializar Git dentro de tu proyecto

Ejemplo: suponiendo que el proyecto ya existe en F:\TESIS\Tensegridad:

```
cd /f/TESIS/Tensegridad
git init
git branch -M main
```

Esto no borra tus archivos. Crea la información interna de Git en la carpeta .git.

Paso 3. Ignorar los archivos temporales de LaTeX

Creá un archivo de texto llamado exactamente .gitignore en la raíz del proyecto, con estas líneas:

```
*.aux
*.log
*.out
*.toc
```

```
*.synctex.gz
*.fls
*.fdb_latexmk
*.xdv
*.bbl
*.blg
```

Así se excluyen los archivos auxiliares habituales. Los .tex, figuras y archivos bibliográficos siguen disponibles para versionar. El PDF final se puede incluir o ignorar según cómo prefieras trabajar.

Paso 4. Crear el primer commit y vincular GitHub

```
git add .
git commit -m "Version inicial de la tesis"
git remote add origin https://github.com/TU_USUARIO/Tesis-Tensegridad.git
git push -u origin main
```

Reemplazá TU_USUARIO por tu cuenta de GitHub. -u establece la vinculación con la rama remota y normalmente solo hace falta la primera vez. Si Git solicita autenticación, iniciá sesión con una cuenta autorizada.

4. Uso diario en cualquiera de las computadoras

Antes de comenzar: traer cambios de GitHub

```
git status
git pull
```

Comprobá que no haya cambios locales pendientes que puedan entrar en conflicto. git pull obtiene e incorpora los commits nuevos.

Cuando terminás: subir cambios a GitHub

```
git add .
git commit -m "Descripcion de los cambios"
git push
```

El add prepara cambios, el commit crea una versión local y el push la envía a GitHub. Si no hay modificaciones, Git puede responder nothing to commit.

Importante: Si trabajás en la raíz del repositorio, git add . incluye archivos nuevos, modificados y eliminados dentro de esa carpeta. git add -A incluye los cambios de todo el repositorio desde cualquier subcarpeta.

5. Ejemplo concreto: TINKATUL y TINKNOTE

En TINKATUL (carpeta existente)

```
cd /f/BLAS/Microbiologia_GIT
git pull
# Trabajar y guardar los archivos en TeXstudio
git add .
git commit -m "Cambios desde TINKATUL"
git push
```

En TINKNOTE (carpeta existente)

```
cd /c/BLAS/Microbiologia
git pull
# Trabajar y guardar los archivos en TeXstudio
git add .
git commit -m "Cambios desde TINKNOTE"
git push
```

Para pasar de una computadora a la otra: primero hacé push desde la primera y luego pull en la segunda. Evitá modificar simultáneamente el mismo archivo sin sincronizar.

6. Comandos de diagnóstico

```
git status                # Estado de los archivos y la rama
git remote -v             # Repositorio remoto vinculado
git branch                # Rama actual (por ejemplo, main)
git log -5 --oneline      # Ultimos cinco commits
git diff                  # Cambios sin preparar
git fetch                 # Actualizar datos del remoto, sin integrar
```

Que git status indique up to date with origin/main no garantiza que GitHub no tenga cambios nuevos: compara contra la última información remota descargada. Ejecutá git fetch para actualizarla, o directamente git pull para incorporar novedades.

7. TeXstudio y recomendaciones prácticas

No necesitás cerrar TeXstudio antes de hacer git pull. Guardá primero tus archivos y prestá atención a cualquier aviso de que un documento abierto cambió externamente; recargalo solo si no tenés ediciones sin guardar.

Antes de un pull, si git status muestra archivos modificados, revisalos. Si aparece un conflicto de fusión, no sobrescribas archivos a ciegas: resolvé el conflicto antes de continuar. Los archivos temporales de compilación suelen mantenerse fuera de Git mediante .gitignore.

Recordatorio: git init inicia el control de versiones de una carpeta local; git clone copia un repositorio remoto existente. Una vez preparado el proyecto, el ciclo habitual es pull → editar → add → commit → push.

Actualización: tesis y privacidad en GitHub

Anexo de octubre de 2026. Complementa la guía anterior y registra la configuración que efectivamente se utilizó.

1. Carpeta y repositorio correctos de la tesis

La carpeta local de trabajo es **F:\TESIS** (no F:\TESIS\Tensegridad, que aparece como ejemplo en la sección 3 de la guía original). El repositorio remoto es **tinkatul/Tesis-Tensegridad** y la rama principal es **main**.

```
cd /f/TESIS
git status
git remote -v
git log -1 --oneline
```

El primer commit ya se publicó en GitHub: **45f6526**, con el mensaje *Version inicial de la tesis*. El estado **HEAD -> main, origin/main** en el historial indica que esas referencias apuntaban al mismo commit al momento de la comprobación; para verificar cambios remotos posteriores, ejecutar git fetch.

2. Convertir un repositorio público en privado

En GitHub: abrir el repositorio > **Settings** > **General** > **Danger Zone** > **Change repository visibility** > **Make private**. Confirmar la operación. No hay que volver a crear el repositorio ni modificar la URL de Git.

La tesis ya quedó configurada como **Private**. Solo el propietario y los usuarios autorizados pueden acceder a ella. El cambio no elimina copias que terceros pudieran haber obtenido mientras estuvo pública.

3. Por qué GitHub muestra martiniaco en el commit

Hay tres conceptos distintos: **propietario** del repositorio (tinkatul), **autor** del commit (nombre y correo registrados por Git) y **cuenta autenticada** para subir cambios. El nombre de la computadora TINKATUL no determina el autor.

El commit inicial de la tesis registra como autor y committer a **Martin Moran** con el correo **moran.martin.andres@gmail.com**. GitHub lo atribuye a **martiniaco** por la asociación de ese correo con dicha cuenta. Esto no significa que el repositorio pertenezca a martiniaco ni es un error de sincronización.

```
git config user.name
git config user.email
git log -1 --format=fuller
```

Si esa atribución no molesta, **no hace falta cambiar nada**. Para cambiar la identidad de futuros commits solo en este repositorio, se pueden utilizar git config --local user.name y git config --local user.email con los valores deseados. Los commits ya creados no se modifican automáticamente.

4. Comandos de trabajo y comprobación

Antes de empezar a editar, revisar los cambios locales y traer los nuevos commits; antes de preparar el commit, inspeccionar qué archivos se agregarán.

```
cd /f/TESIS
git status
git pull
# Editar y guardar los archivos .tex
git status --short
git add -- cap-1.tex   # ejemplo: agregar solo un archivo
git diff --cached --stat
git commit -m "Actualizacion del capitulo 1"
git push
```

Para incluir todos los cambios del proyecto, usar **git add -A** solo después de revisar git status y comprobar que no haya archivos temporales, datos privados u otros elementos que no deban subirse. El archivo .gitignore ayuda a excluir auxiliares de LaTeX.

5. Nota sobre comandos pegados en Git Bash

Ejecutar cada comando en una línea separada. Si se concatena por error el texto de dos comandos, Git puede responder *error: pathspec ... did not match any file(s) known to git*. Ese mensaje, por sí solo, no implica pérdida de archivos.

Anexo: comandos y versiones de Git

Actualización de octubre de 2026. Este anexo complementa la guía previa con los ejemplos utilizados en PLANIALTIS, la tesis y las computadoras TINKATUL/TINKNOTE.

1. Consultar y cambiar nombre y correo de los commits

Consultar los valores globales (no modifica nada):

```
git config --global user.name
git config --global user.email
```

Ver el valor efectivo y el archivo de configuración que lo proporciona:

```
git config --show-origin --get user.name
git config --show-origin --get user.email
```

Cambiar los datos para los futuros commits de este usuario de Windows:

```
git config --global user.name "Martin Moran"
git config --global user.email "correo@ejemplo.com"
```

La opción **--global** aplica a todos los repositorios del usuario, salvo configuraciones locales que la reemplacen. Para cambiar solo un repositorio, entrar a su carpeta y usar **git config --local user.name** y **git config --local user.email**. Los commits anteriores no cambian.

2. Cuenta autenticada de GitHub (distinta del autor)

```
git credential-manager github list
git credential-manager github login --username tinkatul
```

El primer comando enumera las cuentas registradas en Git Credential Manager; el segundo permite iniciar sesión con tinkatul. La identidad de autor y la cuenta utilizada para acceder a repositorios privados son independientes. No borres credenciales de otras cuentas sin necesidad.

3. Git Bash frente a CMD de Windows

Los comandos Git son iguales en ambas terminales: **git status**, **git add**, **git commit**, **git push**, **git pull**. Lo que cambia es la sintaxis de las rutas y los comandos de la terminal.

```
Git Bash: cd "/f/OneDrive - Martin/OneDrive/01 - Scripts/planialtimetrias"
CMD:      cd /d "F:\OneDrive - Martin\OneDrive\01 - Scripts\planialtimetrias"
```

Para verificar la instalación, usar **git --version**. Para saber dónde estás en Git Bash, **pwd**. Para cerrar Git Bash, **exit** o Ctrl+D. En Git Bash, la letra de unidad se escribe normalmente /f/ (no /f:).

4. Guardar cambios: add, commit y push

Desde la carpeta interior PLANIALTIS, el punto de **git add .** se refiere a esa carpeta y sus subcarpetas. Desde la raíz del repositorio, conviene indicar explícitamente PLANIALTIS/ para no preparar archivos ajenos.

```
git status
git add .
git diff --cached --stat
git commit -m "Corregir ubicacion de carteles"
git push
```

Alternativa desde la raíz del repositorio: **git add -A -- PLANIALTIS/**. **git add** prepara cambios, **commit** los registra localmente y **push** envía commits a GitHub. Si no hubo cambios, **git diff --cached --stat** no muestra nada y no hace falta crear un commit.

5. Traer cambios y clonar en otra computadora

Para actualizar un repositorio ya descargado, entrar a su carpeta y ejecutar **git pull**. Antes de hacerlo, comprobar que no haya cambios locales sin guardar.

```
cd /c/GITHUB/PLANIALTIS
git status
git pull
```

Para descargarlo por primera vez en una computadora nueva (la carpeta de destino PLANIALTIS no debe existir):

```
mkdir -p /c/GITHUB
cd /c/GITHUB
git clone https://tinkatul@github.com/tinkatul/PLANIALTIS.git
```

En un repositorio privado, si aparece "Repository not found" aun cuando la página web se abre, revisar qué cuenta está autenticada. La URL con tinkatul@ ayuda a seleccionar la cuenta correcta; no incluye una contraseña.

6. Consultar el historial y recuperar versiones

```
git log --oneline --decorate
git tag
git show V4.71 --stat
git diff V4.71 HEAD --stat
```

git log muestra todos los commits; **git tag** muestra las versiones con nombre (como V4.71). Los comandos anteriores solo consultan y no modifican archivos. Para ver el contenido de un archivo antiguo:

```
git show V4.71:PLANIALTIS_V4_71/LEEME_PRIMERO.txt
```

Para extraer una versión histórica sin reemplazar la carpeta actual, desde la raíz del repositorio:

```
mkdir -p /f/PLANIALTIS_V4_71_RECUPERADA
git archive V4.71 PLANIALTIS_V4_71/ | tar -x -C /f/PLANIALTIS_V4_71_RECUPERADA
```

7. Diferencia entre commit y etiqueta

Cada commit ya tiene un identificador único y conserva un estado del proyecto. No hace falta asignarle un número V4.xx a cada cambio. Una **etiqueta** (tag) sirve para destacar una versión importante o validada.

```
git tag -a V4.72 -m "PLANIALTIS V4.72"
git push origin V4.72
```

Estos comandos se ejecutan cuando el commit correspondiente a V4.72 ya está creado y se desea etiquetarlo. No repetir una etiqueta existente. Para la tesis se utiliza, por ejemplo, V1.00.

8. Subir desde GitHub web y precauciones

También se pueden subir archivos desde la página web del repositorio mediante **Add file > Upload files > Commit changes**. En ese caso el commit se crea directamente en GitHub: antes de seguir trabajando localmente, ejecutar **git pull**. Para cambios numerosos, Git Bash o CMD son más prácticos.

En una computadora corporativa, comprobar las políticas de la empresa antes de clonar repositorios personales o copiar archivos de trabajo. No agregar contraseñas, tokens ni archivos confidenciales a los commits.

Anexo: comandos adicionales y ejemplos

Este anexo complementa la guía anterior, sin repetir sus procedimientos principales. Los ejemplos están pensados para Git Bash, Windows, PLANIALTIS y proyectos LaTeX.

A. Navegar y abrir carpetas en Git Bash

```
pwd
ls
ls -a
cd tesis
cd ..
cd /f/LATEX/neovim
cd /c/Users/moran/tesis
explorer .
code .
```

pwd indica la carpeta actual; **ls** lista archivos y **ls -a** incluye los ocultos, como .gitignore. **cd ..** sube un nivel. **explorer .** abre la carpeta actual en Windows; **code .** la abre en VS Code. En Git Bash, F:\LATEX\neovim se escribe /f/LATEX/neovim. Usá comillas si hay espacios.

B. Consultas de configuración global y local

```
git config --list
git config --local --list
git config --list --show-origin
git config user.name
git config user.email
```

La configuración **local** solo existe dentro de un repositorio y puede prevalecer sobre la global. **--show-origin** indica de qué archivo procede cada valor. Para cambiar solo un proyecto, desde su repositorio:

```
git config --local user.name "Martin Moran"
git config --local user.email "moran.martin.andres@gmail.com"
```

C. Instalar Git desde PowerShell

```
winget install Git.Git
winget install --scope user Git.Git
```

La segunda variante solicita instalación en el ámbito del usuario, cuando el paquete lo admite. En computadoras corporativas, respetá las políticas de instalación y acceso a GitHub.

D. Crear un repositorio privado con GitHub CLI

```
gh repo create PLANIALTIS --private --source=. --remote=origin --push
```

Este comando requiere **GitHub CLI (gh)** instalado, una sesión autenticada mediante **gh auth login** y estar dentro de un repositorio Git local. Crea el repositorio privado, configura origin y publica la rama. Es una **alternativa** al método manual de crear el repositorio en la web y agregar origin; no se deben ejecutar ambos procedimientos sin comprobar primero el remoto.

E. Archivo .gitignore: crear, editar y guardar

```
touch .gitignore
notepad .gitignore
ls -a
git add .gitignore
git commit -m "Agregar gitignore para temporales LaTeX"
git push
```

Ejemplo ampliado de patrones para proyectos LaTeX (colocalos en .gitignore, una línea por patrón):

```
*.aux     *.log       *.out      *.toc
*.lof     *.lot       *.fls      *.fdb_latexmk
*.synctex.gz         *.bbl      *.bcf
*.blg     *.run.xml   *.xdv      *.nav
*.snm     *.vrb
```

Cada patrón debe ir en **su propia línea** dentro de .gitignore. El archivo solo ignora archivos no rastreados; si Git ya los sigue, hay que dejar de rastrearlos por separado.

F. Excluir carpetas específicas de PLANIALTIS

```
printf '/funcionando/\n/superado/\n' > .gitignore
```

Atención: el signo > reemplaza por completo el contenido anterior de .gitignore. Usalo solo al crear ese archivo desde cero; si ya existe, editá y agregá las líneas sin borrar las reglas previas.

G. Comparar archivos y versiones

```
git diff -- secciones/01_dominios.tex secciones/03_morfologia.tex
git diff d63e6b5 793812b
git diff HEAD~1 HEAD
git log
```

El primer comando muestra cambios locales no preparados en dos archivos. El segundo compara dos commits identificados por sus códigos; el tercero compara el commit actual con el anterior. **git log** muestra el historial detallado. Salí del visor con la tecla **q**.

H. Consultar temporalmente una versión anterior

```
git status
git switch --detach d63e6b5
# Examinar archivos de esa version
git switch main
```

Usá este procedimiento **solo con el árbol de trabajo limpio**. Durante la consulta estarás en estado *detached HEAD*; evitá modificar archivos o crear commits allí. Para volver, usá **git switch main** (en tus repositorios la rama es main, no master). Una alternativa sin cambiar de versión es **git show COMMIT:ruta/archivo**. Evitá **git reset** si todavía no conocés sus efectos.

I. Repositorio de Overleaf y cambios entre computadoras

```
git clone https://git@git.overleaf.com/6a05145a88d6748015979733
mv 6a05145a88d6748015979733 tesis
cd tesis
git pull
# Editar archivos LaTeX localmente
git add .
git commit -m "Cambios capitulo 4"
git push
```

El enlace corresponde a un proyecto específico de Overleaf y puede requerir permisos o un plan compatible con integración Git. **mv** renombra la carpeta clonada a tesis. Antes de editar, comprobá **git status** y ejecutá **git pull**. Overleaf y GitHub son remotos distintos: revisá **git remote -v** para saber a cuál enviás los cambios.

J. Agregar archivos concretos y describir commits

```
git add cap-4.tex
git add bibliografia.bib
git commit -m "Agregar analisis de convergencia"
git commit -m "Corregir referencias bibliograficas ASTM"
```

```
git commit -m "Actualizar tablas de desplazamientos"
```

Los tres últimos comandos son **ejemplos alternativos de mensajes**, no una secuencia para ejecutar de corrido. Cada commit debe describir los cambios realmente preparados. Para consultar sin modificar, usá **git status** y **git diff --cached --stat**.

K. Recordatorios de seguridad

Antes de subir, revisá **git status** y los cambios preparados. **git fetch** descarga información remota sin incorporarla a tus archivos; **git pull** la incorpora a tu rama. Evitá publicar archivos privados o corporativos sin autorización. Para un repositorio nuevo, **git remote add origin URL** se ejecuta una sola vez; si origin ya existe, consultalo con **git remote -v**.

Anexo: seis comandos útiles para trabajar con seguridad

Este anexo agrega comandos que no estaban explicados en la guía anterior. Los ejemplos se ejecutan desde Git Bash dentro del repositorio. Antes de descartar cambios, revisá **git status**.

1. Descartar modificaciones no preparadas

```
git restore archivo.tex
```

Devuelve el archivo a su versión del índice (staging area), descartando las modificaciones **no preparadas** de ese archivo. **Precaución:** puede hacerte perder trabajo no guardado en commits; revisá antes con **git diff -- archivo.tex**.

2. Sacar un archivo de la preparación, sin borrar los cambios

```
git restore --staged archivo.tex
```

Retira el archivo del próximo commit, pero conserva sus modificaciones en la carpeta de trabajo. Es útil si hiciste **git add .** y querés excluir un archivo.

3. Guardar cambios temporalmente

```
git stash
```

Aparta temporalmente las modificaciones de archivos rastreados y deja la carpeta de trabajo en un estado más limpio. No equivale a un commit ni a un push. Por defecto, los archivos nuevos no rastreados no se incluyen; para incluirlos, podés usar **git stash -u**.

4. Recuperar los cambios temporales

```
git stash pop
```

Intenta restaurar los cambios guardados y, si se aplican correctamente, quita esa entrada del stash. Puede producir conflictos si el proyecto cambió entretanto. Para consultar qué hay guardado: **git stash list**.

5. Crear una rama y pasar a ella

```
git switch -c prueba-correccion-carteles
```

Crea una rama local nueva y cambia a ella. Sirve para probar modificaciones de PLANIALTIS sin trabajar directamente sobre **main**. Para volver: **git switch main**. Los cambios sin guardar pueden impedir el cambio de rama o acompañarte a la otra rama; verificá **git status**.

6. Consultar ramas locales y remotas

```
git branch -a
```

Muestra las ramas locales y las referencias remotas conocidas. Para actualizar la información remota sin fusionar cambios, ejecutá **git fetch** antes de consultar. Las ramas remotas suelen mostrarse como **remotes/origin/main**.

Ejemplo: corregir lo preparado por error

```
git status
```

```
git add .
git diff --cached --stat
git restore --staged archivo_que_no_quiero.tex
git diff --cached --stat
git commit -m "Actualizar documentos seleccionados"
```

Regla práctica: restore --staged no borra tus ediciones; **restore** sin **--staged** sí puede descartarlas. Si tenés dudas, consultá primero **git status** y **git diff**.

Anexo: crear .gitignore desde Git Bash

Ampliación del **Paso 3** de la guía. Trabajá en la raíz del proyecto LaTeX, donde está **main.tex**.

1. Ubicarse en el proyecto

```
cd /f/BLAS/Microbiologia
pwd
ls -a
```

La ruta es un ejemplo. **pwd** muestra la carpeta actual y **ls -a** permite ver archivos ocultos.

2. Crear y editar .gitignore

```
touch .gitignore
notepad .gitignore
```

Si el archivo ya existe, **touch** no borra su contenido. En el Bloc de notas, pegá estas líneas y guardá con **Ctrl + S**:

```
*.aux
*.log
*.out
*.toc
*.synctex.gz
*.fls
*.fdb_latexmk
*.xdv
*.bbl
*.blg
```

3. Registrar las reglas en Git y subirlas a GitHub

Ejecutá estos cinco comandos, uno por línea, en este orden:

```
git status
git add .gitignore
git diff --cached --stat
git commit -m "Agregar gitignore para archivos temporales LaTeX"
git push
```

git status permite revisar los cambios; **git add .gitignore** prepara ese archivo; **git diff --cached --stat** muestra lo preparado; **git commit** registra el cambio local; **git push** lo publica en el remoto configurado.

Precauciones

Los patrones van sin espacios y uno por línea. **.gitignore** no deja de rastrear archivos que ya estaban versionados: para esos casos hace falta una operación adicional, que conviene revisar antes de ejecutarla. Si el repositorio aún no tiene remoto o rama publicada, puede requerir configuración inicial antes de **git push**.

Anexo: compartir repositorios con otro usuario de GitHub

Este procedimiento sirve para PLANIALTIS, la tesis, Microbiología y otros repositorios. Compartir un repositorio **no exige compartir la contraseña** de tu cuenta.

Caso 1. Invitar a alguien a un repositorio privado

Desde GitHub, con la cuenta propietaria del repositorio:

1. Abrí el repositorio, por ejemplo **tinkatul/PLANIALTIS**.
2. Entrá a **Settings**.
3. En la barra lateral, buscá **Collaborators** o **Collaborators and teams**, dentro de Access (la ubicación puede variar).
4. Elegí **Add people** o **Add collaborator**.
5. Escribí el usuario de GitHub de la otra persona, seleccioná la cuenta correcta y enviá la invitación.
6. La otra persona debe **aceptar la invitación** para obtener acceso.

En un repositorio personal, la invitación de colaborador normalmente concede capacidad de lectura y escritura. Si solo querés que vea el contenido, verificá las opciones de permisos disponibles para ese tipo de repositorio antes de invitarlo; en repositorios de organizaciones existen roles más granulares.

Caso 2. Qué hace la persona invitada

1. Inicia sesión en su propia cuenta de GitHub.
2. Acepta la invitación recibida en GitHub o por correo.
3. En su computadora, abre Git Bash y clona el repositorio con su propia autenticación:

```
cd /c/GITHUB
git clone https://github.com/tinkatul/PLANIALTIS.git
cd PLANIALTIS
git status
git remote -v
```

La carpeta **/c/GITHUB** es un ejemplo; debe existir previamente o crearse con **mkdir -p /c/GITHUB**. Para un repositorio privado, Git puede solicitar autenticación; la cuenta invitada debe ser la que tenga acceso.

Caso 3. Cómo trabajan ambos sobre el mismo repositorio

Antes de editar, cada persona actualiza su copia. Después de guardar los archivos, prepara y publica sus cambios:

```
git status
git pull
# Editar y guardar archivos
git status
git add .
git diff --cached --stat
git commit -m "Describir cambios realizados"
git push
```

La otra persona ejecuta **git pull** para recibir los nuevos commits. Eviten editar simultáneamente los mismos archivos sin coordinarse. Si Git detecta conflictos, hay que resolverlos antes de continuar.

Caso 4. Colaborar sin dar acceso de escritura directo

En un repositorio público, la otra persona puede crear un **fork** en su cuenta, hacer cambios en su copia y abrir un **pull request** hacia tu repositorio. Vos revisás los cambios y decidís si los incorporás. En repositorios privados, los forks y las contribuciones externas dependen de los permisos y las políticas aplicables; no supongas que cualquier usuario puede hacerlo.

Cómo quitar el acceso más adelante

Desde el repositorio: **Settings** → **Collaborators** (o sección equivalente) → localizar al usuario → **Remove access**. Esto impide futuros accesos autorizados, pero no borra las copias que la persona haya descargado previamente.

Recordatorio: dar acceso a un repositorio GitHub no equivale a compartir una carpeta de OneDrive; los cambios se intercambian mediante commits y sincronización. No compartas tokens, claves ni contraseñas.

Anexo: seguimiento, preparación y comparación en Git

Este anexo amplía la sección «Comparar archivos y versiones» de la guía. Los ejemplos se ejecutan desde Git Bash dentro de la carpeta del repositorio.

1. «Bajo seguimiento» no significa «preparado»

Tracked (bajo seguimiento): Git ya conoce el archivo porque forma parte de un commit anterior o porque lo agregaste con git add. **Untracked (sin seguimiento):** es un archivo nuevo que Git todavía no incorporó. **Staged (preparado):** es el contenido exacto que se incluirá en el próximo commit.

Al clonar un repositorio, los archivos descargados ya están bajo seguimiento. No es necesario volver a incorporarlos como nuevos. Pero si los modificás, sí necesitás preparar esas modificaciones para el siguiente commit.

2. Por qué se repite git add

```
git status
git diff -- main.tex           # Ver cambios aun no preparados
git add main.tex               # Preparar la version actual
git diff --cached -- main.tex  # Ver lo que entrara al commit
git commit -m "Actualizar main.tex"
```

Si modificás main.tex **después** de git add, repetí git add para incluir esas últimas modificaciones. git commit -m por sí solo registra únicamente lo que ya estaba preparado. Como atajo, git commit -am prepara y registra cambios de archivos ya seguidos, pero no incorpora archivos nuevos.

3. Comandos de comparación

Comando	Qué permite consultar
git status	Archivos modificados, nuevos, eliminados y cambios preparados.
git diff	Cambios de contenido todavía no preparados con git add.
git diff --stat	Resumen por archivo de los cambios no preparados.
git diff --word-diff	Diferencias palabra por palabra (no preparadas).
git diff --check	Detecta problemas de espacios en blanco en cambios no preparados.
git diff --cached	Diferencias ya preparadas respecto del último commit.
git diff --cached --stat	Resumen por archivo de los cambios preparados.

El separador **--** antes de una ruta indica que lo siguiente es un nombre de archivo: git diff -- main.tex. No modifica el archivo.

4. Integrar archivos de otra computadora sin perder cambios

Primero cloná el repositorio y conservá la carpeta vieja como respaldo. Compará las dos carpetas (por ejemplo, con WinMerge) **antes de sobrescribir** archivos si ambas contienen modificaciones valiosas. Si copiás encima de la carpeta clonada, Git puede mostrar las diferencias respecto del último commit, pero no combina automáticamente los contenidos.

```
git status
git diff --stat
git diff -- main.tex
# Revisar, elegir o combinar cambios y guardar
git add .
git diff --cached --stat
git commit -m "Consolidar versiones"
git push
```

Atención: git diff no muestra por defecto el contenido de archivos nuevos sin seguimiento; git status sí los identifica. Una vez preparados con git add, podés examinarlos con git diff --cached. La versión anterior registrada en Git sigue disponible en el historial, pero los cambios que existían solo en una carpeta no versionada requieren conservar esa

carpeta como respaldo.

5. Git también funciona sin GitHub

Git permite controlar versiones enteramente en una computadora, sin conexión a Internet. En una carpeta que todavía no es repositorio:

```
git init
git add .
git commit -m "Primera version"
git log --oneline
```

GitHub es opcional: sirve como repositorio remoto para compartir, respaldar y sincronizar entre computadoras. Sin remoto, los commits y el historial permanecen en la carpeta .git local; conviene tener una copia de seguridad.

Anexo: historial completo, espacio y planos DWG

Esta sección complementa la guía de Git y GitHub. Todos los comandos se ejecutan desde Git Bash, dentro de la carpeta del repositorio.

1. Consultar todas las versiones registradas

```
git log
git log --oneline
git log --oneline --graph --all --decorate
git log -10 --oneline
```

git log muestra commits, autores, fechas y mensajes. **--oneline** resume cada commit en una línea; **--graph --all --decorate** muestra la estructura de ramas y referencias; **-10** limita a los diez más recientes. Para salir de la vista paginada, presioná **q**.

```
a8f32c1 Corregir perfil longitudinal
b71d09e Actualizar carteles PRH
c93a512 Modificar parametros del ducto
```

La secuencia inicial de caracteres es el identificador abreviado del commit. Se utiliza para consultar o recuperar una versión concreta.

2. Ver qué se modificó en un commit o archivo

```
git show --stat a8f32c1
git show --name-only a8f32c1
git show a8f32c1
git log --oneline -- Plano_Gasoducto.dwg
```

--stat lista archivos y resume cambios; **--name-only** muestra solo los nombres. **git show** permite ver el contenido del cambio cuando es un formato comparable (por ejemplo, TXT, LSP o TEX). Para DWG, Git no muestra diferencias gráficas útiles. El historial por archivo permite identificar los commits que afectaron a ese DWG.

3. Dónde está el historial y cuánto espacio ocupa

Git guarda los commits y objetos históricos en la carpeta oculta **.git**, situada en la raíz del repositorio. En Windows, activá **Ver** → **Mostrar** → **Elementos ocultos** y consultá las **Propiedades** de esa carpeta.

```
du -sh .git
git count-objects -vH
```

du -sh .git indica el tamaño total aproximado. **git count-objects -vH** detalla objetos y paquetes internos. La carpeta .git puede crecer aunque el tamaño de los archivos actuales permanezca estable. No la borres: contiene el historial local.

4. Usar Git sin GitHub

```
cd /ruta/de/tu/proyecto
git init
git add .
git commit -m "Version inicial"
```

Los commits se guardan localmente. Podés registrar varias versiones y consultarlas sin Internet ni **git push**. GitHub es opcional: sirve para alojar una copia remota y sincronizar computadoras. Varios commits seguidos de un solo push publican los mismos commits que hacer un push después de cada uno, si no intervienen otros cambios remotos.

5. ¿Conviene versionar planos DWG?

Sí, especialmente para conservar hitos de un proyecto (por ejemplo: antes de cambiar la traza, después de corregir el perfil o al emitir una revisión). Los DWG son **binarios**: Git puede guardar y recuperar versiones, pero

no comparar su geometría mediante git diff ni fusionar fácilmente ediciones simultáneas. Para comparar dibujos, usá la herramienta **DWG Compare** de AutoCAD.

```
git add Plano_Gasoducto.dwg
git commit -m "Corregir perfil longitudinal"
```

No es necesario crear un commit después de cada Ctrl+S: conviene hacerlo al terminar etapas importantes. Cada commit conserva una versión identificable. Con muchos DWG grandes, el repositorio puede crecer bastante; para ese caso puede evaluarse Git LFS, especialmente si se va a usar un servidor remoto.

6. DWG, BAK y SV$: diferencias importantes

DWG es el dibujo guardado. Con **ISAVEBAK = 1**, AutoCAD normalmente mantiene un **.bak** con el estado anterior al último guardado; ese archivo se reemplaza en guardados posteriores. Los **.sv$** son archivos de guardado automático orientados a recuperación ante fallos: pueden eliminarse al cerrar normalmente y no constituyen un historial fiable. Git, en cambio, conserva las versiones que registraste explícitamente con commits.

Precaución: Git no reemplaza una copia de seguridad independiente. Si el historial está solo en el mismo disco y el disco falla, también podés perderlo. Evitá incluir archivos temporales (.bak, .sv$, .dwl, .dwl2) en los commits; podés excluirlos con .gitignore. Si el DWG pertenece a una empresa o cliente, respetá sus políticas antes de subirlo a un servicio remoto.

7. Ejemplo de rutina local para planos

```
git status
git add Plano_Gasoducto.dwg
git commit -m "Revision antes de modificar la traza"
# Trabajar y guardar normalmente en AutoCAD...
git add Plano_Gasoducto.dwg
git commit -m "Traza y perfil corregidos"
git log --oneline -- Plano_Gasoducto.dwg
```

Con esta rutina tenés un historial consultable en tu computadora. Si querés capturar versiones por horario sin intervención, necesitás automatización adicional o un sistema de copias históricas: Git no crea commits por sí solo.

Anexo: etiquetas (tags) y recuperación de versiones

Este apartado amplía el historial de Git con un ejemplo real de PLANIALTIS. Los commits registran estados del proyecto; los tags permiten ponerles nombres estables, como V4.72 o V4.73.

1. Diferencia entre commit, mensaje y tag

Cada commit tiene un identificador único (hash), un mensaje descriptivo y una fecha. El tag es una referencia adicional, opcional; escribir “V4.73” en el mensaje no crea automáticamente el tag.

```
60d76a9  version 4.73 - Evita volver a pedir TN y DUCTO
f04f428  version V4.72 - PTRESTARURATRAMOPRH
```

2. Consultar las etiquetas existentes

```
git tag
git tag --list "V4.*"
git log --oneline --decorate
```

En el ejemplo, git tag mostraba únicamente V4.71, aunque el historial ya incluía los commits V4.72 y V4.73.

3. Asignar tags a commits anteriores

```
git tag V4.72 f04f428
git tag V4.73 60d76a9
git tag
```

Estos comandos crean etiquetas locales sin modificar archivos ni crear commits nuevos. Si una etiqueta ya existe, Git avisará y no la reemplazará automáticamente.

4. Crear el tag al terminar una nueva versión

```
git add .
git commit -m "Version V4.74 - Descripcion de cambios"
git tag V4.74
```

Sin indicar un hash, git tag V4.74 señala el commit actual (HEAD). También podés crear una etiqueta anotada, con mensaje propio:

```
git tag -a V4.74 -m "Version estable V4.74"
```

5. Enviar commits y etiquetas a GitHub

```
git push
git push origin V4.74
```

Para enviar todas las etiquetas locales que falten en el remoto:

```
git push origin --tags
```

Importante: un git push común no envía necesariamente todos los tags. Los tags también funcionan sin GitHub, dentro del repositorio local.

6. Consultar una versión anterior sin tocar la actual

La opción más segura para inspeccionar una versión antigua es crear una carpeta de trabajo adicional. Primero, verificá que el tag exista con git tag.

```
git worktree add --detach ../PLANIALTIS_V4.72 V4.72
```

Esto crea una carpeta hermana con el estado de V4.72. Tu rama principal permanece intacta. Al terminar, cerrá los archivos abiertos en esa carpeta y ejecutá:

```
git worktree remove ../PLANIALTIS_V4.72
```

No borres manualmente una carpeta worktree si podés retirarla con el comando anterior. Si hay cambios sin guardar, Git puede impedir la eliminación para protegerlos.

7. Recuperar un solo archivo desde una versión

Si necesitás recuperar un archivo concreto, podés traerlo desde el tag. Atención: reemplaza el contenido de ese archivo en tu carpeta de trabajo; revisá y guardá antes cualquier cambio pendiente.

```
git restore --source=V4.72 -- PLANIALTIS.lsp
git diff -- PLANIALTIS.lsp
```

La ruta del archivo debe ser relativa a la raíz del repositorio. En tu proyecto puede estar dentro de una subcarpeta; verificá su ubicación con git status o el Explorador.

8. Ver a qué commit apunta un tag

```
git show --no-patch V4.73
git rev-list -n 1 V4.73
```

9. Precauciones importantes

Un tag no es una copia física separada: señala un commit conservado en el historial. Conviene usar nombres únicos por versión y evitar mover tags ya publicados. Para recuperar un estado completo sin modificar main, preferí git worktree frente a cambiar de versión directamente en la carpeta principal.

Resumen rápido

Objetivo	Comando
Ver tags	git tag
Etiquetar commit anterior	git tag V4.72 f04f428
Etiquetar el commit actual	git tag V4.74
Subir una etiqueta	git push origin V4.74
Consultar etiquetas en historial	git log --oneline --decorate
Abrir versión en otra carpeta	git worktree add --detach ../PLANIALTIS_V4.72 V4.72

Anexo: worktrees, archivos ocultos y limpieza

Este anexo amplía la recuperación de versiones anteriores de PLANIALTIS y explica los mensajes observados en Git Bash al retirar una copia temporal.

1. ¿Por qué aparecen .git y .gitignore?

.git: en el repositorio principal suele ser una carpeta que guarda los objetos, referencias e historial de Git. En un *worktree* adicional es normalmente un **archivo pequeño** que apunta a los datos administrativos del repositorio principal. No es una copia del historial.

.gitignore: es un archivo ordinario del proyecto, normalmente versionado, que define qué archivos no rastreados debe ignorar Git. Si existía en el commit recuperado, también aparecerá en el worktree.

Aunque un .gitignore incluya la regla **.gitignore**, Git seguirá rastreándolo si ya estaba versionado. Para reglas locales no compartidas puede usarse **.git/info/exclude**. En general conviene mantener .gitignore versionado.

2. ¿Dónde se crea la versión recuperada?

```
git worktree add --detach ../PLANIALTIS_V4.72 f04f428
```

Desde la carpeta **planialtimetrias**, la ruta **../** significa subir un nivel: la nueva carpeta PLANIALTIS_V4.72 se crea al lado del repositorio principal, no dentro. Es preferible para evitar mezclar archivos recuperados con los actuales.

```
git worktree list
```

Permite ver las carpetas de trabajo registradas. La indicación **(detached HEAD)** significa que esa copia apunta a un commit concreto y no está trabajando sobre la rama main.

3. Eliminar correctamente un worktree

Primero cerrá AutoCAD, editores, Explorador y cualquier programa que utilice archivos de la copia recuperada. Después, desde el repositorio principal:

```
git worktree remove ../PLANIALTIS_V4.72
git worktree list
```

Si el comando pregunta repetidamente **Should I try again? (y/n)** y la carpeta sigue bloqueada, respondé **n**; no conviene insistir indefinidamente. Una causa posible en Windows es un bloqueo de archivos por OneDrive, antivirus o algún programa abierto.

4. Mensajes de error habituales

fatal: not a git repository: el comando se ejecutó fuera del repositorio. Volvé con **cd planialtimetrias** y repetí la consulta.

validation failed ... .git does not exist: el worktree puede haber desaparecido físicamente, pero seguir registrado en los metadatos del repositorio.

prunable en **git worktree list**: Git detecta un registro de worktree cuyo directorio ya no está disponible o no es válido. Si realmente fue eliminado y no es una unidad temporal desconectada, podés limpiar su registro:

```
git worktree prune
git worktree list
```

Permission denied al eliminar **.git/worktrees/.../logs**: Windows impidió limpiar parte de los metadatos. Detené los reintentos, cerrá los programas que accedan a la carpeta y revisá el estado con **git worktree list**. No borres la carpeta principal .git.

Si **git worktree list** ya muestra solo el repositorio principal, la copia adicional dejó de estar registrada, aunque pudiera quedar algún directorio administrativo residual. No es necesario forzar su borrado para seguir trabajando.

Anexo: comprobaciones y commits pendientes

5. Verificar el estado después de una limpieza

```
git status
git log -1 --oneline
git worktree list
```

Un resultado como **On branch main** y **nothing to commit, working tree clean** indica que no hay cambios pendientes en el directorio de trabajo ni en el área de preparación. No significa que todo esté subido a GitHub.

Si aparece **Your branch is ahead of origin/main by 1 commit**, hay un commit local aún no publicado, siempre según la última información remota conocida por Git. No es un error.

6. Varios commits y un solo push

```
git add .
git commit -m "Version V4.73"
# Seguir trabajando y registrar otros cambios...
git add .
git commit -m "Version V4.74"
# Subir cuando corresponda:
git push
```

El **push** publica todos los commits locales pendientes de la rama, no solo el último. Podés postergarlo; los commits ya existen en tu computadora. Hasta publicarlos, no estarán disponibles en GitHub ni en otros equipos. Si el remoto cambió entretanto, quizá necesites sincronizar primero.

7. Historial y etiquetas no se pierden al quitar el worktree

Eliminar una copia creada con **git worktree add** no elimina el commit original ni sus tags. La versión V4.72 puede recuperarse de nuevo por el hash **f04f428** o por el tag **V4.72** si fue creado.

```
git tag
git log --oneline --decorate
git worktree add --detach ../PLANIALTIS_V4.72 f04f428
```

Consejo: para una comprobación rápida antes de publicar, usá **git status**. Si la rama está adelantada y el árbol de trabajo está limpio, podés dejar el push para más adelante sin perder los commits locales.

Anexo: recuperar versiones y mantener main

Ejemplos basados en PLANIALTIS. Antes de recuperar una version, comproba que estas en el repositorio correcto y que el identificador o tag existe: **git tag** o **git log --oneline**.

1. Consultar una version anterior sin cambiar main

```
git worktree add --detach ../PLANIALTIS_V4.72 V4.72
```

Crea una carpeta separada apuntando al commit de V4.72, sin crear una rama nueva. Al indicar un tag o un hash, **git worktree add** normalmente tambien utiliza detached HEAD aunque omitas **--detach**. La rama main no cambia.

2. Desarrollar una alternativa desde una version antigua

```
git worktree add -b prueba-v4.72 ../PLANIALTIS_PRUEBA V4.72
```

Crea la rama **prueba-v4.72** desde V4.72 y la abre en otra carpeta. Los commits nuevos quedan en esa rama; **main** permanece intacta hasta una integracion deliberada.

3. Restaurar V4.72 como contenido vigente de main, conservando el historial

Esta es la opcion preferible si queres deshacer una version defectuosa pero mantener rastreables todos los commits anteriores.

```
git status
git restore --source=V4.72 --staged --worktree :/
git status
git commit -m "Restaurar contenido de PLANIALTIS V4.72"
```

El nuevo commit pasa a ser el vigente de **main**, con el contenido de los archivos versionados de V4.72. La V4.73 sigue en el historial. Si el arbol recuperado coincide con el actual, Git no creara un commit de cambios por este procedimiento.

Precaucion: git restore sobrescribe cambios locales de los archivos afectados, incluidos cambios preparados. No limpia automaticamente todos los archivos no seguidos o ignorados. Revisalos por separado antes de restaurar.

4. Por que revisar git status antes

```
git status
```

Si aparece **nothing to commit, working tree clean**, no hay modificaciones rastreadas pendientes ni archivos nuevos no ignorados pendientes. Si hay cambios, guardalos en un commit o respaldalos antes de restaurar. Los archivos ignorados no necesariamente aparecen en status.

5. Retroceder main directamente: opcion de mayor riesgo

```
git reset --hard V4.72
```

Mueve **main** al commit V4.72 y descarta cambios locales de archivos rastreados. Los commits posteriores dejan de estar en la historia de main, aunque pueden seguir recuperables por otras referencias o temporalmente por reflog. **No usar a la ligera**, especialmente si esos commits ya se publicaron en GitHub: podria requerir reescribir el historial remoto.

Resumen: que comando elegir

Objetivo	Comando principal
Ver version antigua en otra carpeta	git worktree add --detach ...
Crear rama desde version antigua	git worktree add -b ...
Volver al contenido anterior sin perder historial	git restore ... + git commit
Mover main hacia atras (riesgoso)	git reset --hard ...

6. Guardar esta guia con Git usando siempre el mismo nombre

Mantene **Guia_Git_GitHub.pdf** en un repositorio propio. Cuando recibas una edicion nueva, reemplaza el PDF anterior y registra el cambio. Asi evitas archivos con nombres crecientes y podes recuperar cualquier edicion mediante el historial.

```
git add Guia_Git_GitHub.pdf
git commit -m "Actualizar guia: recuperacion de versiones"
# git push  (cuando quieras sincronizar con GitHub)
```

Git puede conservar distintas versiones del PDF, pero al ser binario no permite comparar sus parrafos mediante un diff textual habitual. Si algun dia queres comparar cambios de redaccion, podes guardar tambien el documento fuente editable.

Anexo: iniciar un repositorio y elegir master o main

Este apartado responde a las consultas sobre el primer commit de la guia PDF. Los comandos son validos para cualquier repositorio local, incluido PLANIALTIS.

1. ¿Funciona git init sin git branch -M main?

```
git init
```

Si. Git crea el repositorio y prepara una rama inicial. Su nombre depende de la configuracion de Git: puede ser **master**, **main** u otro. No hay diferencia funcional entre master y main: ambos son nombres de rama.

```
git status
git branch --show-current
```

Antes del primer commit, git status permite ver el nombre de la rama inicial. Tambien podes elegirlo explicitamente al crear el repositorio:

```
git init -b main
# O, si preferis:
git init -b master
```

2. ¿Para que sirve git branch -M main?

```
git branch -M main
```

Renombra la rama actual a main, incluso si ya existiera una rama local con ese nombre (por eso se usa -M). **No es obligatorio.** Si preferis conservar master, no ejecutes este comando. Si algun tutorial usa main, reemplazalo por master cuando el nombre de la rama sea parte del comando.

3. Interpretar el primer commit de la guia

```
git commit -m "Primera version de la guia"
[master (root-commit) 0843fcc] Primera version de la guia
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Guia_Git_GitHub.pdf
```

master: rama en la que quedo registrado el commit. **root-commit:** primer commit, sin padre. **0843fcc:** identificador abreviado del commit. **create mode 100644:** Git agrego un archivo normal al repositorio. **0 insertions, 0 deletions:** es esperable para un PDF, ya que Git no contabiliza lineas de texto en archivos binarios como en un .txt.

4. Actualizar la guia sin cambiar el nombre ni la rama

Conserva el archivo **Guia_Git_GitHub.pdf** con ese mismo nombre. Reemplazalo por la nueva edicion dentro de la carpeta del repositorio y ejecuta:

```
git status
git add Guia_Git_GitHub.pdf
git commit -m "Actualizar guia: git init y rama master"
```

No necesitas crear otra rama ni hacer git push para conservar el nuevo commit localmente. Si mas adelante conectas el repositorio con GitHub, podras publicar la rama master con el nombre que elegiste.

5. Consultas frecuentes

¿Tengo que usar main porque GitHub lo usa en ejemplos? No. GitHub acepta repositorios con rama master. El nombre de la rama predeterminada del repositorio remoto se puede configurar por separado.

¿Puedo cambiar de nombre despues? Si. Renombrar una rama local no borra sus commits; si ya existe una rama remota publicada, habra que coordinar tambien el cambio en el remoto.

¿Que pasa si el PDF cambia pero Git indica 0 inserciones y 0 eliminaciones? No es un error. El PDF es binario: Git registra que cambio el archivo y guarda su contenido, pero no muestra un diff textual convencional.

¿Como recupero una edicion anterior de la guia? Consulta el anexo anterior sobre git worktree, git restore y git reset. Esos procedimientos tambien funcionan en una rama llamada master: solo cambia el nombre de la rama cuando corresponda.

Anexo - Ignorar carpetas con .gitignore

El archivo **.gitignore** indica qué archivos y carpetas sin seguimiento debe omitir Git. Se coloca habitualmente en la raíz del repositorio y se puede crear en cualquier momento.

1. Reglas para carpetas

```
# Carpeta pruebas en cualquier nivel
pruebas/

# Solo la carpeta temporales de la raiz
/temporales/

# Carpetas de copias de seguridad
copias_seguridad/

# Todos los archivos .bak
*.bak
```

La barra final **/** indica que la regla corresponde a un directorio. La barra inicial **/** ancla la regla al directorio que contiene ese .gitignore (en este ejemplo, la raíz). Sin barra inicial, **pruebas/** coincide con carpetas de ese nombre en distintos niveles.

2. Si la carpeta ya estaba registrada

Agregarla a .gitignore **no** hace que Git deje de seguir los archivos previamente incorporados al repositorio. Para retirarlos del índice sin borrarlos del disco:

```
git rm -r --cached pruebas/
git add .gitignore
git commit -m "Ignorar carpeta de pruebas"
```

El commit también registrará que esos archivos dejan de estar presentes en la versión actual del repositorio, aunque permanecen en el historial anterior.

3. Comprobar las reglas

```
git status
git check-ignore -v pruebas/archivo.txt
```

git status permite verificar qué cambios aparecen. **git check-ignore -v** identifica la regla que ignora una ruta; para archivos ya seguidos por Git, se puede usar **git check-ignore -v --no-index**.

4. Precauciones

Git no registra carpetas vacías por sí solas. Un .gitignore no borra archivos del disco ni elimina versiones históricas. Si solo querés ignorar archivos nuevos, no hace falta ejecutar **git rm --cached**. Para la guía PDF, un .gitignore sigue siendo opcional si el repositorio contiene únicamente el PDF.

Anexo - Consultar versiones y regresar a la actual

Caso práctico: tenés la última edición de **Guia_Git_GitHub.pdf** y querés abrir una edición anterior solo para consultarla, sin crear commits ni cambiar de rama.

1. Consultar los commits disponibles

```
git log --oneline
git status
```

Conviene verificar que no haya modificaciones locales pendientes. Los comandos de restauración pueden sobrescribir trabajo sin guardar.

2. Traer temporalmente una versión anterior

```
git restore --source=75d6ebf --worktree -- :/
```

Git copia a la carpeta de trabajo el contenido de los archivos **registrados** tal como estaban en ese commit. **HEAD** y la rama actual (por ejemplo, **master**) siguen apuntando al último commit. No se crea un commit nuevo.

Significado: **--source=75d6ebf** elige la versión; **--worktree** modifica los archivos físicos; **--** separa opciones de rutas; **:/** abarca todo el repositorio, desde su raíz.

3. Leer la versión antigua y volver a la última

```
git restore --source=HEAD --worktree -- :/
```

Esto restaura los archivos registrados desde el commit vigente de **master**. Si solo abriste el PDF para leerlo, recuperás así la última versión sin **git add**, **git commit** ni **git push**.

También puede utilizarse **git restore -- :/** cuando el área de preparación (staging) coincide con HEAD. Si hiciste **git add** durante la consulta, preferí la variante explícita con **--source=HEAD** para elegir la versión del último commit.

4. Recuperar un solo archivo o todos

```
git restore --source=75d6ebf -- Guia_Git_GitHub.pdf
git restore --source=75d6ebf --worktree -- :/
```

El primer comando restaura únicamente el PDF. El segundo restaura todos los archivos registrados del repositorio. La ruta **.** se refiere a la carpeta actual y sus descendientes; **:/**, a la raíz del repositorio y todo su contenido registrado.

5. Si querés que la versión antigua sea la vigente

```
git restore --source=75d6ebf --staged --worktree -- :/
git commit -m "Restaurar version anterior"
```

En este caso, **--staged** prepara la recuperación para el commit y **--worktree** actualiza la carpeta. El nuevo commit conserva el historial previo. Es distinto de una consulta temporal.

Precaución: estos comandos pueden descartar cambios locales en archivos registrados. No eliminan automáticamente archivos sin seguimiento (*untracked*).

Anexo - Entender git commit -m

¿Qué significa -m?

```
git commit -m "Actualizar guia"
```

```
git commit --message "Actualizar guia"
```

-m es la abreviatura de **--message**: indica que el texto entre comillas es el mensaje del commit. Ambas formas son equivalentes.

¿Qué pasa si omitís -m?

```
git commit "adbas"
```

Git interpreta **adbas** como una ruta o nombre de archivo, no como el mensaje. Si esa ruta no existe, normalmente aparece el error **pathspec ... did not match any file(s) known to git**.

```
git commit
```

Sin un mensaje indicado en la línea de comandos, Git abre el editor configurado para que lo escribas. El commit solo se crea si hay cambios aptos para confirmar y se proporciona un mensaje válido.

Recordatorio: tres áreas distintas

HEAD: último commit de la rama. **Staging:** contenido preparado con git add. **Worktree:** archivos que ves y abrís en Windows. **git restore** modifica staging y/o worktree; por sí solo no crea commits.

Anexo actualizado · Octubre de 2026

Este anexo amplía la guía con el caso real de PLANIALTIS: migración de la raíz del repositorio, comparación entre Git local y GitHub, normalización de finales de línea y publicación de commits acumulados.

1. Dónde está el repositorio y cómo reconocerlo

El directorio **.git/** determina la raíz del repositorio. **.gitignore** contiene reglas para excluir archivos no deseados. Para PLANIALTIS, ambos deben estar dentro de la carpeta PLANIALTIS, junto a PLANIALTIS.bundle/.

```
git rev-parse --show-toplevel
git status -sb
git remote -v
```

Si el primer comando termina en **planialtimetrias/PLANIALTIS**, la raíz es la esperada. Evitá ejecutar comandos desde la antigua carpeta padre, que todavía podría conservar otro .git.

2. Estado local y estado remoto: no son lo mismo

```
git log --oneline -5             # últimos commits locales
git log origin/main --oneline -5 # últimos commits remotos conocidos
git fetch origin                # actualizar datos desde GitHub
git ls-remote origin refs/heads/main  # consultar GitHub en vivo
```

origin/main es una referencia almacenada localmente; no se actualiza automáticamente. **git fetch** descarga la información nueva sin cambiar los archivos de trabajo. **git ls-remote** necesita conexión y consulta directamente el servidor.

3. Cómo interpretar -s, -b y -u

```
git status -sb
# -s = short (formato breve)
# -b = branch (información de la rama)

git push -u origin main
# -u = --set-upstream (vincula main con origin/main)
```

Después de configurar el upstream, generalmente alcanza con **git push** y **git pull**, sin repetir el nombre de la rama y del remoto.

4. Comparar qué falta subir o descargar

```
git fetch origin
git status -sb
git log origin/main..main --oneline  # commits solo locales
git log main..origin/main --oneline  # commits solo remotos
```

Ejemplo: **## main...origin/main [ahead 4]** indica cuatro commits locales pendientes de subir. Si aparece **behind** hay commits remotos pendientes de incorporar; si aparece **diverged** ambas ramas avanzaron por separado y se debe revisar antes de hacer push.

5. Trabajar sin conexión y subir después

```
git add .
git commit -m "PLANIALTIS V4.78"
# Se pueden crear más commits sin Internet.

# Cuando vuelva la conexión:
git fetch origin
git status -sb
git push
```

Git guarda los commits en .git aunque cierres Git Bash o apagues la computadora. Al hacer push, se envían juntos todos los commits que falten. Si el servidor avanzó por su cuenta, primero hay que reconciliar los cambios: no usar --force como solución automática.

6. Migración real de PLANIALTIS: resumen del procedimiento

Se partió de un repositorio cuya raíz era **planialtimetrias/**, con el proyecto dentro de **planialtimetrias/PLANIALTIS/**. El objetivo fue que la raíz Git coincidiera con la carpeta PLANIALTIS, sin cambiar su ruta de trabajo y conservando el historial.

Se generó y verificó un respaldo completo del historial con **git bundle**; luego se creó un clon local de migración, se trasladaron los archivos del subdirectorio a su raíz y se registró la reorganización en un nuevo commit.

```
git bundle create ../PLANIALTIS_respaldo.bundle --all
git bundle verify ../PLANIALTIS_respaldo.bundle
git clone --no-hardlinks . ../PLANIALTIS_MIGRACION
```

El clon local recibió inicialmente un remoto origin que apuntaba al repositorio local de origen. Se corrigió con **git remote set-url origin https://github.com/tinkatul/PLANIALTIS.git**. No confundir un clon local con un clon hecho directamente desde GitHub.

Antes del traslado definitivo se compararon los archivos, se guardó una copia adicional de PLANIALTIS y se colocaron .git y .gitignore del repositorio migrado dentro de la carpeta habitual. El antiguo repositorio de la carpeta padre se conservó temporalmente por seguridad.

7. Finales de línea LF y CRLF en Windows

Con **core.autocrlf=true**, Git puede advertir que LF será convertido a CRLF. Un listado de archivos modificados después de trasladar .git no implica necesariamente cambios reales en el código. Antes de descartar nada, comparar el contenido:

```
git diff --ignore-space-at-eol --exit-code
echo $?
# 0: sin diferencias bajo ese criterio
# 1: hay diferencias; revisarlas
```

En el caso de PLANIALTIS, esa comparación devolvió 0. Después se ejecutó **git add --renormalize .**, que reevalúa la normalización en el índice. Se verificó que **git diff --cached --stat** y **git status --short** quedaran vacíos. No hacer restore indiscriminado sobre archivos de trabajo sin respaldo.

8. Publicación verificada en GitHub

El remoto tenía como último commit **f04f428**. Tras recuperar la conexión, se ejecutó un push normal que publicó cuatro commits pendientes, incluida la reorganización **c5d186d** y la versión **V4.77 (16a9d5e)**.

```
git fetch origin
git log origin/main..main --oneline
git push -u origin main
# Confirmación observada:
# f04f428..16a9d5e  main -> main
# branch main set up to track origin/main
```

No se necesitó **git push --force**. Un mensaje **forced update** durante fetch puede indicar que cambió de forma no lineal una referencia remota conocida; no significa, por sí solo, que el usuario haya forzado una publicación.

9. Comprobaciones y limpieza posterior

```
git status -sb
git rev-parse --show-toplevel
git remote -v
git log -1 --oneline
```

Antes de retirar el antiguo **planialtimetrias/.git**, verificar que el repositorio nuevo funciona en PLANIALTIS, que GitHub muestra el último commit y que los respaldos están accesibles. No eliminar a ciegas las carpetas de respaldo

ni el archivo .bundle. Si existe un repositorio Git anidado en la carpeta padre, los comandos ejecutados desde ella pueden apuntar al repositorio equivocado.

10. Referencia rápida

```
git status -sb                    # estado resumido
git log --oneline -5               # historial local
git fetch origin                   # actualizar referencias remotas
git log origin/main..main --oneline # pendientes de subir
git log main..origin/main --oneline # pendientes de incorporar
git push                           # publicar commits
git ls-remote origin refs/heads/main # consulta remota directa
```

Regla práctica: commit guarda una versión en tu computadora; fetch consulta y descarga datos remotos; push publica tus commits; pull incorpora cambios remotos a tu rama local.

Capítulo adicional: ramas, detached HEAD y worktrees

Ejemplos de la guía **Guia_Git_GitHub.pdf**. Este capítulo complementa los anteriores y explica cómo consultar versiones antiguas sin sobrescribir la versión actual. Los comandos se ejecutan desde Git Bash, dentro del repositorio principal, salvo que se indique otra cosa.

1. Tres conceptos que conviene distinguir

Concepto	Qué representa	Dónde se ve
Commit	Instantánea registrada del proyecto, identificada por un hash.	git log --oneline
Rama (branch)	Nombre móvil que apunta al último commit de una línea de trabajo.	git branch -v
Worktree	Carpeta de trabajo adicional que comparte el historial del repositorio.	git worktree list

Importante: una rama no es una carpeta; y un worktree no es una rama. Podés tener varias ramas en una sola carpeta, o varias carpetas de trabajo vinculadas a un mismo repositorio.

2. Consultar y cambiar de rama

```
git branch
git branch --show-current
git branch -v
git switch master
```

El asterisco de **git branch** señala la rama activa. **git branch --show-current** devuelve el nombre de la rama actual, pero queda vacío si estás en detached HEAD. **git switch master** vuelve a la rama master; si tu repositorio usa main, reemplazá master por main.

3. Crear ramas de pruebas y eliminarlas

```
git switch master
git switch -c pruebas
git branch
git switch master
git branch -d pruebas
```

-c equivale a **--create** en git switch: crea una rama y cambia a ella. **-d** equivale a **--delete** en git branch y protege frente a la eliminación de ramas con commits no integrados. **-D** fuerza el borrado y debe evitarse si no se comprobó el historial. No se puede eliminar la rama actualmente activa; primero cambiá a otra.

4. Detached HEAD: mirar un commit sin estar en una rama

```
git switch --detach 75d6ebf
git status -sb
git branch
git switch master
```

En detached HEAD, HEAD apunta directamente al commit **75d6ebf**, no a una rama. Es útil para consultar una versión histórica, pero **no es un modo de solo lectura**: se pueden modificar archivos y crear commits. Si hacés commits que querés conservar, creá una rama antes de abandonar ese estado, por ejemplo **git switch -c rescate**.

Precaución: git switch puede reemplazar los archivos visibles de la carpeta de trabajo para mostrar el commit elegido. Git protege los cambios locales incompatibles, pero los archivos no seguidos o ignorados requieren especial atención. Comprobá **git status** antes de cambiar.

5. Extraer solamente un PDF antiguo

```
git show 75d6ebf:Guia_Git_GitHub.pdf > Guia_Git_GitHub_V2.pdf
```

Ejecutado en **Git Bash**, guarda el PDF histórico con otro nombre sin cambiar la rama ni los archivos actuales. La ruta después de los dos puntos debe ser la ruta del archivo dentro del repositorio en ese commit. Para archivos binarios, evitá trasladar sin más esta redirección a versiones antiguas de Windows PowerShell, donde el tratamiento de la salida puede alterar los bytes.

Carpetas de trabajo adicionales con git worktree

6. Abrir un commit antiguo en una carpeta independiente

```
git worktree add --detach ../GUIA_VERSION_2 75d6ebf
git worktree add --detach ../GUIA_VERSION_3 0843fcc
git worktree list
```

El argumento **../GUIA_VERSION_2** indica una carpeta hermana de la carpeta principal. **--detach** evita asociar el worktree a una rama: la copia queda posicionada en el commit histórico. Se pueden abrir los PDF de las tres carpetas y compararlos sin alternar la versión visible de la carpeta principal.

Carpeta	Posición	Propósito
04 - Guia	master	Versión actual y trabajo cotidiano
GUIA_VERSION_2	75d6ebf (detached HEAD)	Consulta histórica
GUIA_VERSION_3	0843fcc (detached HEAD)	Otra consulta histórica

7. Worktree asociado a una rama

```
git worktree add ../GUIA_PRUEBAS pruebas
```

Esta variante usa una rama existente llamada **pruebas**. Normalmente Git impide tener la misma rama activa en dos worktrees a la vez, para evitar que dos carpetas intenten administrar simultáneamente el mismo puntero de rama. En cambio, se puede abrir el commit al que apunta master en otra carpeta sin asociarlo a master:

```
git worktree add --detach ../GUIA_COPIA master
```

Si omitís **--detach** al pasar un hash o referencia de commit, Git puede crear una rama nueva, según los argumentos y las reglas de git worktree add. Para una copia puramente histórica, usá --detach de forma explícita.

8. Retirar un worktree sin perder los commits

```
git worktree remove ../GUIA_VERSION_2
git worktree remove ../GUIA_VERSION_3
git worktree list
```

Primero cerrá los PDF, el Explorador y las terminales ubicadas dentro de esas carpetas. El comando elimina la carpeta adicional y sus metadatos de worktree, pero **no elimina los commits** ni la rama principal. Si hay modificaciones locales, Git puede negarse a eliminarla: revisá el contenido antes de considerar opciones forzadas.

9. Errores de permisos de Windows y OneDrive

Si aparece **Unlink of file failed**, **Permission denied** o **Should I try again? (y/n)**, cerrá los archivos abiertos, dejá de usar la carpeta desde otras terminales y esperá o pausá OneDrive. Si el reintento no funciona, respondé **n** y comprobá el estado:

```
git worktree list
git status -sb
```

No borres manualmente **.git/worktrees**. Si el directorio de un worktree fue eliminado pero quedó su registro, **git worktree prune** puede limpiar metadatos obsoletos; usalo solo tras confirmar que esa carpeta ya no existe y que no se trata de una unidad temporal desconectada. Si la carpeta física sigue existiendo pero Git ya no la registra, revisá su contenido antes de borrarla manualmente.

Sincronización, recuperación y guía rápida

10. Consultar local, remoto conocido y GitHub en vivo

```
git status -sb
git log --oneline -5
git log origin/main --oneline -5
git fetch origin
git ls-remote origin refs/heads/main
```

origin/main es una referencia remota almacenada localmente; no se consulta GitHub en vivo cada vez que se lee. **git fetch origin** la actualiza sin fusionar cambios ni modificar el árbol de trabajo. **git ls-remote** consulta directamente la referencia de GitHub sin descargar los objetos de los commits.

11. Qué falta subir o incorporar

```
git log origin/main..main --oneline
git log main..origin/main --oneline
git status -sb
```

El primer comando muestra commits locales ausentes de la referencia remota conocida; el segundo, commits remotos conocidos que no están en main local. Si la rama se llama **master**, reemplazá main por master. Un estado **[ahead 3]** indica tres commits locales por delante de la referencia remota conocida; **[behind 2]** indica dos por detrás. Para una comparación actualizada, ejecutá antes **git fetch origin**.

12. Qué hace -u al publicar

```
git push -u origin master
# En repositorios cuya rama se llama main:
git push -u origin main
# Después, si la relación de seguimiento ya existe:
git push
```

-u es **--set-upstream**: establece el seguimiento entre la rama local y la rama remota. No cambia el nombre de la rama ni convierte master en main. Si otra computadora ya publicó cambios, revisá el historial y sincronizá antes de intentar subir; no uses **--force** para resolverlo sin analizarlo.

13. OneDrive y la carpeta .git

OneDrive puede copiar archivos de **.git**, pero Git no garantiza la integridad de su base de datos cuando un sincronizador replica parcialmente archivos internos o cuando dos equipos trabajan al mismo tiempo. La estrategia recomendada es **un repositorio Git local por equipo** y sincronización de commits mediante GitHub. Evitá compartir una misma carpeta .git activa mediante OneDrive. Si actualmente la compartís, primero realizá un respaldo y verificá ambos historiales antes de reorganizar carpetas.

14. Lista de verificación antes de cambiar o eliminar algo

```
git rev-parse --show-toplevel
git status -sb
git branch -v
git worktree list
git log --oneline -5
```

Estas consultas ayudan a identificar el repositorio, los cambios pendientes, las ramas, los worktrees y los últimos commits. Son una buena primera respuesta cuando un archivo parece haber desaparecido después de cambiar de versión.

15. Recordatorio para esta guía

El archivo oficial debe llamarse siempre **Guia_Git_GitHub.pdf**. Los nombres alternativos de los ejemplos son solo copias históricas temporales, no el nombre del documento principal. El historial de Git registra las distintas versiones aunque el archivo conserve siempre el mismo nombre.

Anexo: referencia y flujos de Git

Ampliación de la guía existente. Los ejemplos se ejecutan desde Git Bash, dentro del repositorio correspondiente. Los valores HASH, NOMBRE y RUTA son marcadores que debés sustituir. Este anexo no reemplaza los capítulos anteriores.

Cómo consultar este anexo

A. Índice alfabético: cuando conocés el nombre del comando. B. Comandos por función: cuando sabés qué querés hacer. C. Secuencias de trabajo: comandos en el orden recomendado. D. Opciones frecuentes y precauciones.

Reglas previas a operaciones delicadas

```
git rev-parse --show-toplevel
git status -sb
git log --oneline --decorate -8
git branch -vv
```

Antes de cambiar de rama, restaurar archivos o reescribir historial, asegurate de que no existan modificaciones locales que necesites conservar. Un commit de respaldo o una rama temporal ayudan a evitar pérdidas.

A. Índice alfabético de comandos

Comando	Uso principal
git add	Prepara altas, modificaciones y eliminaciones para el próximo commit.
git branch	Lista, crea, renombra o elimina ramas.
git cherry-pick	Aplica el cambio de un commit específico sobre la rama actual.
git clone	Descarga un repositorio y crea una copia local.
git commit	Registra los cambios preparados en un nuevo commit.
git config	Consulta o cambia la configuración de Git.
git diff	Compara modificaciones, commits o ramas.
git fetch	Actualiza las referencias remotas conocidas sin fusionarlas.
git init	Crea un repositorio local en la carpeta actual.
git log	Muestra el historial de commits.
git ls-files	Lista archivos registrados en el índice.
git ls-remote	Consulta referencias disponibles en el remoto.
git merge	Integra otra línea de desarrollo en la rama actual.
git pull	Obtiene e integra cambios remotos.
git push	Publica commits o referencias en el remoto.
git reflog	Muestra movimientos locales recientes de HEAD y referencias.
git remote	Administra nombres y direcciones de remotos.

Comando	Uso principal
git reset	Mueve HEAD o una rama y, según el modo, ajusta índice y archivos.
git restore	Restaura archivos desde el índice o un commit.
git revert	Crea cambios inversos a uno o varios commits sin borrar historial.
git rev-parse	Consulta datos internos, incluida la raíz del repositorio.
git show	Examina el contenido o los cambios de un commit.
git stash	Guarda temporalmente modificaciones no confirmadas.
git status	Muestra el estado de archivos y rama.
git switch	Cambia de rama o entra temporalmente a un commit.
git tag	Crea, lista o elimina etiquetas de versión.
git worktree	Gestiona carpetas de trabajo adicionales de un repositorio.

B. Comandos agrupados por función

Identificar dónde estás

Comando	Qué hace
git rev-parse --show-toplevel	Raíz del repositorio
git status -sb	Rama y cambios pendientes
git branch -vv	Ramas locales y seguimiento
git remote -v	Direcciones de GitHub

Crear y conectar repositorios

Comando	Qué hace
git init	Inicializar una carpeta
git clone URL	Clonar desde GitHub
git remote add origin URL	Vincular remoto
git push -u origin main	Primera publicación y seguimiento

Archivos y registro

Comando	Qué hace
ls -la	Listado físico; comando de Git Bash, no de Git
git ls-files	Archivos registrados
git add RUTA	Preparar archivos concretos
git add -u	Preparar modificaciones y eliminaciones registradas
git commit -m "Mensaje"	Crear commit

Consultar y comparar

Comando	Qué hace
git log --oneline --decorate	Historial breve
git show HASH	Detalle de un commit
git diff	Cambios aún no preparados
git diff --staged	Cambios preparados
git diff HASH1 HASH2	Comparar dos versiones

GitHub y sincronización

Comando	Qué hace
git fetch origin	Actualizar referencias remotas
git pull	Traer e integrar cambios
git push	Publicar commits
git status -sb	Detectar ahead/behind

Ramas

Comando	Qué hace
git switch -c prueba	Crear y entrar en rama
git switch main	Volver a main
git merge prueba	Fusionar prueba en rama actual
git branch -d prueba	Eliminar rama ya fusionada

Versiones y recuperación

Comando	Qué hace
git tag V1.0	Etiquetar el commit actual
git switch --detach HASH	Consultar versión anterior
git restore --source=HASH --staged --worktree :/	Restaurar árbol registrado
git revert HASH	Deshacer un commit mediante otro
git reset --hard HASH	Retroceder rama; operación destructiva

Trabajo paralelo y rescate

Comando	Qué hace
git worktree add --detach RUTA HASH	Abrir versión en otra carpeta
git stash push -m "temporal"	Guardar trabajo sin commit
git reflog	Buscar posiciones recientes
git cherry-pick HASH	Recuperar un cambio específico

C. Secuencias prácticas por función

1. Guardar cambios y subirlos a GitHub

```
git status -sb
git add Neovim/
git diff --staged --stat
git commit -m "Actualizar Neovim"
git push
```

Usá una ruta concreta para no incluir accidentalmente otras carpetas. git add . incorpora los cambios bajo el directorio actual; git add -A, todos los cambios del repositorio.

2. Comenzar a trabajar en otra computadora

```
git clone URL_DEL_REPOSITORIO
cd NOMBRE_DEL_REPOSITORIO
git status -sb
```

Si el repositorio ya está clonado, no vuelvas a clonarlo encima: entrá en la carpeta y usá git fetch o git pull según corresponda.

3. Revisar diferencias entre local y GitHub

```
git fetch origin
git status -sb
git log --oneline origin/main..main
git log --oneline main..origin/main
```

La primera lista muestra commits locales pendientes de publicar; la segunda, commits remotos pendientes de integrar. origin/main es la referencia remota conocida localmente, actualizada por fetch.

4. Consultar una versión anterior y regresar

```
git status -sb
git log --oneline
git switch --detach HASH
# Consultar o probar los archivos
git switch main
```

Detached HEAD significa que no estás trabajando sobre una rama. Para desarrollar desde esa versión, creá una rama nueva con git switch -c NOMBRE HASH.

5. Marcar y publicar una versión

```
git tag -a V1.0 -m "Version validada"
git push origin V1.0
git tag --list
```

Una etiqueta queda apuntando al commit elegido. No avanza con los commits posteriores. Las etiquetas locales no se publican automáticamente con un git push normal.

6. Rama experimental y fusión

```
git switch main
git switch -c pruebas
# Editar archivos
git add .
git commit -m "Ensayo"
```

```
git switch main
git merge pruebas
git push
```

Si main no avanzó, merge puede hacer fast-forward. Si ambas ramas modificaron zonas incompatibles, habrá que resolver conflictos.

C. Recuperación y cambios de rumbo

7. Deshacer un commit conservando el historial: git revert

```
git status -sb
git log --oneline
git revert HASH
git push
```

git revert HASH crea un nuevo commit que invierte los cambios introducidos por ese commit. No equivale a volver al estado completo de HASH. Si el commit que se revierte es una fusión, puede necesitarse la opción -m con el número de padre; no aplicarla sin revisar.

8. Volver al contenido de una versión antigua, con un commit nuevo

```
git switch main
git status -sb
git restore --source=HASH_ANTIGUO --staged --worktree :/
git status -sb
git diff --cached --stat
git commit -m "Restaurar estado de version anterior"
git push
```

Este procedimiento restaura el estado de los archivos registrados en el commit elegido, incluyendo altas y eliminaciones necesarias dentro del repositorio. Los archivos no registrados que ya existen en tu carpeta no se eliminan en general. Revisá siempre el estado y el diff antes del commit. Es la alternativa más sencilla cuando querés que el contenido antiguo vuelva a ser el vigente sin reescribir main en GitHub.

9. Abandonar un camino y continuar desde un commit antiguo

Ejemplo: main llegó a D, pero preferís retomar B. Primero guardá D con una etiqueta o rama y publicá esa referencia; después elegí cómo continuar.

```
A -- B -- C -- D     (main antes)
     \-- E -- F      (main despues)
          C -- D      (camino archivado)
```

Si necesitás que main apunte literalmente al nuevo camino sin un commit de restauración, tendrás que mover o reemplazar la referencia de main. Si main ya se publicó, esto implica reescritura del historial remoto. Para un flujo sencillo y seguro, preferí el procedimiento 8 y mantené las etiquetas como hitos históricos.

10. Guardar un camino antes de retroceder una rama

```
git status -sb
git tag -a camino_descartado_01 -m "Camino anterior"
git push origin camino_descartado_01
# Solo despues de verificar el respaldo, y si corresponde:
git reset --hard HASH_ANTIGUO
```

ATENCIÓN: el último comando descarta modificaciones locales registradas sin confirmar y mueve la rama local. No ejecutes git push --force-with-lease sin revisar el estado remoto y sin entender el impacto en otros clones. Para evitar reescrituras, usá el procedimiento 8.

11. Abrir una versión histórica en otra carpeta

```
git worktree add --detach ../proyecto_anterior HASH
git worktree list
# Al terminar, desde el repositorio principal:
git worktree remove ../proyecto_anterior
```

Ideal para comparar versiones sin sustituir los archivos de tu carpeta habitual. No elimines manualmente la carpeta de un worktree activo.

D. Opciones y parámetros habituales

Opción	Comando(s)	Significado en contexto
-a	ls; git branch; git tag	En ls: incluir ocultos; en branch: ramas locales y remotas; en tag: no es una opción general equivalente.
-A	git add	Preparar todas las altas, modificaciones y eliminaciones.
-b	git checkout (histórico)	Crear rama al cambiar; hoy suele preferirse git switch -c.
-c	git switch	Crear y cambiar a una rama nueva.
-d / -D	git branch	Eliminar rama fusionada / forzar eliminación local.
-l	ls	Listado detallado de archivos.
-m	git commit; git tag; git branch	Mensaje de commit o etiqueta; renombrar rama en branch.
-u	git push; git add	En push: establecer upstream; en add: actualizar solo archivos ya registrados.
-v / -vv	git remote; git branch	Ver direcciones remotas; en branch -vv, seguimiento y último commit.
--all	git branch; git tag; git fetch	El significado exacto depende del subcomando.
--detach	git switch; git worktree	Usar un commit sin asociarlo a una rama activa.
--force-with-lease	git push	Permitir reescritura remota solo si la referencia esperada coincide.
--hard	git reset	Mover referencia, índice y archivos registrados; descarta cambios.
--no-commit	git revert	Aplicar reversión al índice y archivos sin crear todavía el commit.
--oneline	git log	Mostrar cada commit en una sola línea.
--source	git restore	Elegir commit o referencia desde donde recuperar archivos.
--staged	git diff; git restore	En diff: comparar preparados; en restore: actuar sobre el índice.
--worktree	git restore	Restaurar archivos en el directorio de trabajo.

Sintaxis que suele confundirse

```
ls -la                # Git Bash, NO "git ls -la"
git ls-files          # Archivos que Git registra
git status -sb        # Resumen con rama
git log --oneline     # Commits abreviados
git branch -vv        # Seguimiento remoto
```

En Git, las opciones no tienen significado universal. Por ejemplo, -m en git commit indica el mensaje, mientras que -m en git branch permite renombrar una rama.

Glosario mínimo

Repositorio: proyecto e historial administrados por Git. Rama: referencia móvil a un commit. Commit: instantánea registrada. Tag: referencia fija a un punto del historial, salvo modificación explícita. HEAD: posición actual. Índice/staging: selección de cambios para el siguiente commit. Remoto: repositorio externo, por ejemplo GitHub.

Advertencia sobre OneDrive

Es preferible sincronizar commits entre equipos mediante GitHub y evitar usar OneDrive como mecanismo de sincronización de una carpeta .git activa en varios equipos. Antes de mover .git o reorganizar repositorios, verificá la raíz con git rev-parse --show-toplevel y el estado con git status -sb.